DOCUMENTAÇÃO DO PROJETO

SISTEMA DE GERENCIAMENTO DE ESTACIONAMENTO

Disciplina: Programação para Dispositivos Móveis II

Aluno: Celso Sebastião
Turma: 5º Semestre
Professor: Mario
Data: 29/10/2026

Relatório técnico e descritivo do sistema de cadastro e gerenciamento de proprietários e veículos.

1. INTRODUÇÃO

Opresente trabalhotem como objetivo desenvolver um sistema de gerenciamento de estacionamento, permitindo o cadastro e o controle de proprietários e veículos por meio de um aplicativo móvel.

A proposta é facilitar a organização das informações de um estacionamento, substituindo controles manuais por um sistema informatizado. Dessa forma, torna-se possível registrar os dados dos proprietários, cadastrar veículos e relacionar cada veículo ao seu respectivo proprietário.

O projeto é dividido em três partes principais: o aplicativo móvel, o back-end e o banco de dados. Essas partes trabalham em conjunto para permitir que o usuário consulte e gerencie as informações por meio de uma interface.

2. OBJETIVOS

2.1. Objetivo geral

Desenvolver um aplicativo móvel para gerenciamento de proprietários e veículos de um estacionamento, utilizando tecnologias de desenvolvimento mobile, uma API e um banco de dados relacional.

2.2. Objetivos específicos

- Criaruma interface simpleseintuitiva para utilização do sistema.
- Permitir o cadastro, a consulta, a edição e a exclusão de proprietários.
- Permitir o cadastro, a consulta, a edição e a exclusão de veículos.
- Relacionar cada veículo ao seu respectivo proprietário.
- Armazenar as informações em um banco de dados MySQL.
- Desenvolver uma API para a comunicação entre o aplicativo e o servidor.
- Organizar o código em arquivos e pastas com responsabilidades definidas.

3. DESCRIÇÃO DO SISTEMA

O sistemade gerenciamento deestacionamento centraliza as informações relacionadas aos proprietários e aos veículos cadastrados. Por meio do aplicativo, o usuário pode acessar as áreas de proprietários e veículos, consultar registros existentes e realizar operações de cadastro, atualização e exclusão.

O aplicativo é responsável pela interface com o usuário; o back-end recebe e processa as solicitações; e o banco de dados armazena as informações de forma estruturada.

4. TECNOLOGIAS UTILIZADAS

4.1. React Native

O React Native é utilizado para desenvolver a interface do aplicativo móvel com componentes reutilizáveis e código JavaScript. No projeto, ele é responsável pelas telas, formulários, listas e ações disponíveis ao usuário.

4.2. Node.js

O Node.js permite executar JavaScript no ambiente do servidor e é utilizado para executar a aplicação responsável pelo processamento das solicitações.

4.3. Express

O Express é um framework para Node.js que facilita a criação de servidores e APIs. Ele organiza as rotas usadas para consultar, cadastrar, atualizar e excluir registros.

4.4. MySQL

O MySQL é um sistema de gerenciamento de banco de dados relacional. Ele armazena as informações dos proprietários e veículos e permite estabelecer relacionamentos entre as tabelas.

4.5. Sequelize

O Sequelize é uma biblioteca ORM que facilita o acesso ao banco de dados a partir do JavaScript, permitindo representar tabelas por meio de modelos e executar operações sobre os registros.

4.6. JavaScript e TypeScript

O JavaScript é utilizado na lógica do servidor e em partes do aplicativo. Quando utilizado, o TypeScript acrescenta tipagem aos dados e ajuda a identificar determinados erros durante o desenvolvimento.

4.7. API REST

A API REST define os caminhos e métodos HTTP utilizados na comunicação entre o aplicativo e o servidor. As respostas podem ser representadas em JSON, um formato adequado para transmitir objetos e listas de dados.

5. ARQUITETURA DO PROJETO

A arquitetura separa as responsabilidades de cada parte do sistema. O aplicativo envia uma requisição à API; o servidor recebe e processa a solicitação; o Sequelize realiza as operações necessárias no banco de dados MySQL; e o servidor retorna uma resposta ao aplicativo.

Fluxo principal: Usuário → Aplicativo React Native → API Node.js/Express → Sequelize → Banco de dados MySQL.

Essa separação facilita a manutenção e permite que a interface seja modificada sem reescrever toda a lógica de acesso aos dados.

6. FUNCIONALIDADES DO SISTEMA

6.1. Tela inicial

A tela inicial apresenta o acesso às principais áreas do sistema, facilitando a navegação entre proprietários e veículos.

6.2. Cadastro de proprietários

Permite inserir os dados necessários para identificar os proprietários. Os campos previstos incluem nome e CPF. Após o preenchimento, os dados são enviados ao servidor para armazenamento.

6.3. Listagem de proprietários

Apresenta os proprietários cadastrados e permite consultar os registros existentes.

6.4. Edição e exclusão de proprietários

A edição permite alterar os dados de um proprietário. A exclusão remove o registro, respeitando as regras de relacionamento com os veículos associados.

6.5. Cadastro de veículos
Permite registrar placa, ano, valor da mensalidade e proprietário relacionado. A associação permite identificar a pessoa responsável por cada veículo.

6.6. Listagem de veículos
Apresenta os veículos registrados e permite consultar informações como placa, ano, mensalidade e proprietário.

6.7. Edição e exclusão de veículos
A edição atualiza os dados de um veículo, enquanto a exclusão remove o registro do sistema por meio da API.

7. MODELAGEM DO BANCO DE DADOS

7.1. Tabela Proprietário
A tabela de proprietários armazena os dados das pessoas cadastradas. Os campos principais previstos são:

- id_proprietario: identificador único do proprietário.
- nome: nome do proprietário.
- cpf: CPF do proprietário.

7.2. Tabela Veículo
A tabela de veículos armazena as informações dos veículos cadastrados. Os campos principais previstos são:

- id_veiculo: identificador único do veículo.
- placa: placa do veículo.
- ano: ano do veículo.
- mensalidade: valor mensal associado ao veículo.
- fk_proprietario: identificador do proprietário relacionado.

7.3. Relacionamento entre as tabelas
O relacionamento é do tipo um para muitos (1:N): um proprietário pode estar relacionado a um ou mais veículos, enquanto cada veículo possui um proprietário associado conforme a estrutura proposta. A chave estrangeira fk_proprietario estabelece essa ligação.

8. DESENVOLVIMENTO DO BACK-END

O back-end recebe as solicitações do aplicativo e executa operações sobre os dados. Node.js e Express criam o servidor e definem as rotas da API. O Sequelize intermedeia as operações sobre os modelos de proprietários e veículos, e o MySQL mantém os registros armazenados.

As principais responsabilidades do back-end são receber requisições HTTP, processar os dados enviados, consultar registros, cadastrar novos dados, atualizar e excluir registros, retornar respostas e tratar erros.

9. DESENVOLVIMENTO DO APLICATIVO MOBILE

OaplicativodesenvolvidocomReact Nativeapresenta as telasde navegação e gerenciamento dos registros. Os componentes visuais exibem dados, recebem informações dos formulários e disponibilizam ações ao usuário. A comunicação com a API permite enviar informações ao servidor e recuperar os registros armazenados.

A separação do código em componentes, constantes e hooks, quando aplicada, contribui para reduzir repetições e separar a interface da lógica de acesso aos dados.

10. ORGANIZAÇÃO DOS ARQUIVOS

Aorganizaçãodos arquivos tem como objetivotornaroprojeto mais compreensível e facilitar futuras alterações. A pasta components pode reunir componentes visuais reutilizáveis; constants pode armazenar configurações e valores constantes; e hooks pode concentrar funções reutilizáveis relacionadas à lógica da aplicação.

No back-end, os arquivos do servidor, modelos e rotas devem permanecer organizados conforme a estrutura implementada no projeto.

11. COMUNICAÇÃO ENTRE O APLICATIVO E A API

A comunicaçãoocorre por meiode requisições HTTP.Oaplicativoenviaos dadosnecessários para a rota correspondente e o servidor retorna uma resposta. Os métodos mais comuns são:

- GET: consultar informações.
- POST: cadastrar novos registros.
- PUT: atualizar registros existentes.
- DELETE: excluir registros.

Para que a comunicação funcione corretamente, o aplicativo deve utilizar o endereço do servidor adequado ao ambiente em que está sendo executado.

12. TESTES DO SISTEMA

Os testes permitem verificar se as funcionalidades atendem aos requisitos do projeto. As verificações recomendadas incluem:

- Cadastrar um proprietário e confirmar que ele pode ser consultado.
- Consultar a lista de proprietários pela API.
- Editar os dados de um proprietário e verificar a atualização.
- Verificar a exclusão de proprietário e o tratamento de veículos associados.
- Cadastrar um veículo com os dados e o proprietário correspondente.
- Consultar, editar e excluir veículos.
- Reiniciar a aplicação e verificar se os registros permanecem armazenados.

Os resultados registrados devem corresponder aos testes realmente executados no ambiente do projeto.

13. RESULTADOS ESPERADOS

Espera-se queosistema permitaorganizar asinformações de proprietários e veículos em um único ambiente, facilitando o gerenciamento dos registros. O banco de dados relacional permite manter os dados estruturados e estabelecer relacionamentos entre as entidades. Os resultados efetivos devem ser confirmados por meio da execução do aplicativo, da API e da consulta ao banco de dados.

14. DIFICULDADES E POSSÍVEIS MELHORIAS

Durante o desenvolvimento, podem surgir dificuldades relacionadasàconfiguração do ambiente, conexão com o banco de dados, comunicação entre dispositivos e tratamento de erros. Também é importante validar os dados enviados pelos formulários antes de armazená-los.

Como melhorias futuras, o sistema poderá incluir:

- Pesquisa de proprietários por nome ou CPF.
- Pesquisa de veículos por placa.
- Validação de CPF e placa e prevenção de cadastros duplicados.
- Mensagens de confirmação antes de excluir registros.
- Autenticação de usuários e controle de permissões.
- Relatórios, histórico de alterações e melhorias de acessibilidade.

15. CONCLUSÃO

O desenvolvimento dosistema de gerenciamento de estacionamento permite aplicar conceitos importantes de programação para dispositivos móveis, desenvolvimento de APIs e armazenamento de dados. O React Native possibilita criar uma interface móvel, Node.js e Express processam as solicitações, o Sequelize facilita o acesso ao banco de dados e o MySQL armazena as informações. A integração dessas tecnologias demonstra a importância da divisão de responsabilidades em uma aplicação e contribui para o aprendizado de desenvolvimento mobile e back-end. A conclusão sobre o funcionamento completo do sistema deve ser baseada nos testes realizados.











17. REFERÊNCIAS

REACT NATIVE. Documentação oficial. Disponível em: https://reactnative.dev/.

NODE.JS. Documentação oficial. Disponível em: https://nodejs.org/.

EXPRESS. Documentação oficial. Disponível em: https://expressjs.com/.

MYSQL. Documentação oficial. Disponível em: https://dev.mysql.com/doc/.

SEQUELIZE. Documentação oficial. Disponível em: https://sequelize.org/.

Data de acesso: 29/10/2026.